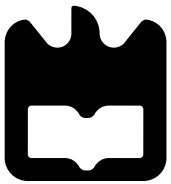


universidade de aveiro

deti

departamento de eletrónica,
telecomunicações e informática

Distributed Systems

Processes, Threads and Concurrency

Eurico Pedrosa <efp@ua.pt>

António Rui Borges

2^{nd} semester - 2025′26

Processe

- A **process** is a **program in execution** managed by the operating system
- Each process has its own **process context**, including:
  - CPU register values
  - Memory mappings (address space)
  - Open files and privileges
- The operating system ensures **isolation** between processes so they cannot interfere with each other’s execution

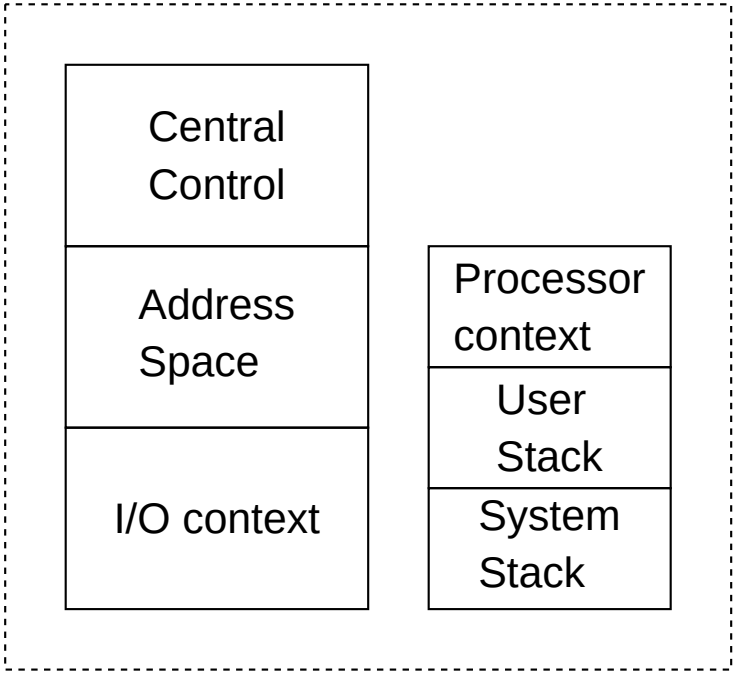
Thread

- A **thread** is a **single flow of control** within a process
- Like a process, a thread executes code independently
- Unlike processes, threads **do not have their own address space**
- Threads maintain only minimal state, mainly the **processor context**
  - e.g., program counter, stack pointer

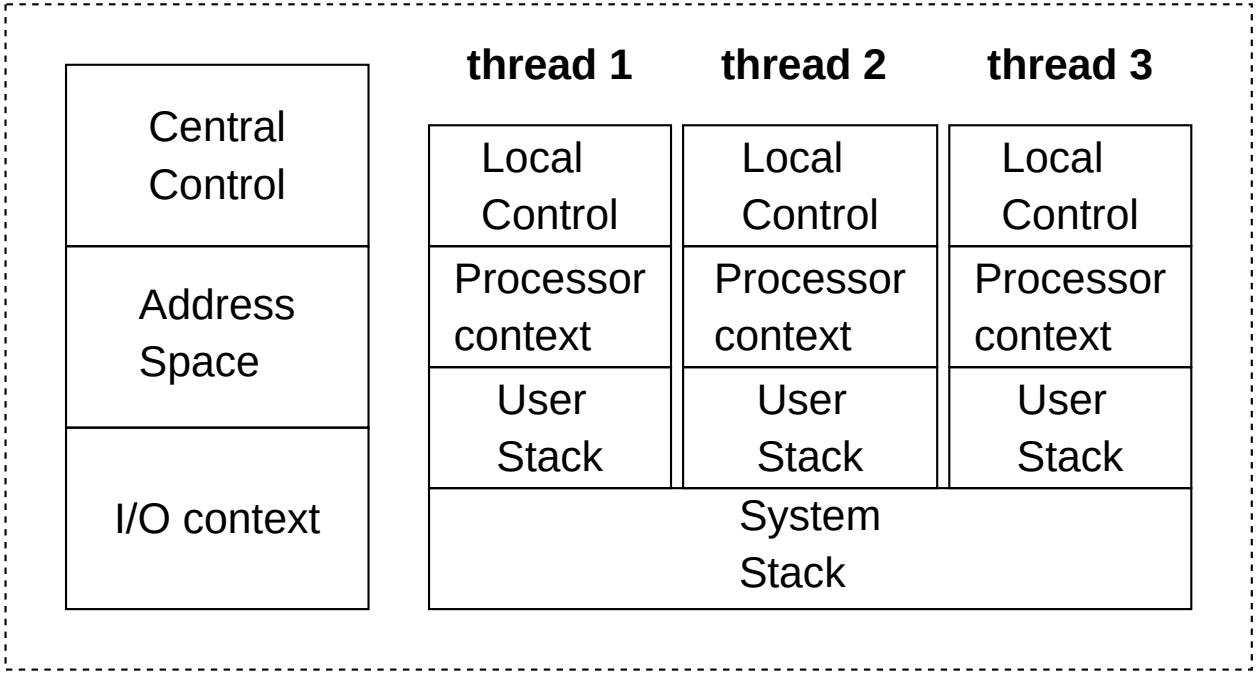
Processes and Threads

- **Threads exist inside processes** and share the process's resources.
- Memory and data are **shared among threads** of the same process.
- Conceptual containment:
  - processor context $\in$ thread context $\in$ process context
- **Processes provide protection and isolation**
- **Threads provide fine-grained concurrency and performance**

Single Threading



Multithreading



Processes Alone Are Not Enough

- Processes provide **strong isolation**, but at a **coarse granularity**
- Creating a process requires:
  - Allocating a new address space
  - Initializing memory segments
  - Setting up independent execution state
- Communication between processes relies on **interprocess communication (IPC)**
  - IPC typically requires **kernel intervention**, leading to extra overhead

Applications requiring many concurrent activities, **process-based designs become inefficient and hard to scale**

Cost of Processes vs Threads

- **Process context switches** are expensive:
  - Switch from user mode to kernel mode
  - Change memory mappings (MMU updates)
  - Flush translation caches (TLB)
- **Thread context switches** are cheaper:
  - Often involve only saving and restoring CPU registers
  - May occur entirely in user space
- As a result:
  - Process switching prioritizes **safety and isolation**
  - Thread switching prioritizes **performance and responsiveness**

Threads Are Crucial in Distributed Systems

- Distributed systems frequently involve:
  - Long communication delays
  - Blocking I/O operations
  - Multiple concurrent interactions
- Threads allow **blocking system calls without blocking the entire process**
- This enables:
  - Handling multiple clients simultaneously
  - Overlapping communication and computation
  - Structuring clients and servers as concurrent activities
- Threads make it possible to retain **simple sequential programming** while still achieving **parallelism and high performance**

Multithreaded Clients

- Distributed clients often operate over **wide-area networks**
  - where communication delays can be high
- Establishing connections and receiving data are **blocking by nature**
- Using multiple threads allows a client to:
  - Initiate communication and continue with other work
  - Maintain several logical connections simultaneously
- Threads help **hide communication latency** while preserving a simple, sequential programming model

Multithreaded Clients

Example: Multithreaded Web Client

- A Web document often consists of multiple components (HTML, images, icons)
- A multithreaded browser:
  - Fetches the main HTML page first
  - Starts separate threads to retrieve other components in parallel
- Each thread performs blocking I/O independently
- The user can interact with the page while remaining data is still being fetched

Multithreaded Servers

- The main use of threads in distributed systems is on the **server side**
- Servers must handle many concurrent client requests efficiently
- A multithreaded server:
  - Allows multiple requests to be processed in parallel
  - Avoids blocking the entire server when one request waits for I/O
- Threads significantly improve **throughput and responsiveness**
  - Even on single-processor systems.

Multithreaded Servers

Dispatcher/Worker Server Model

- A common server organization uses:
  - One **dispatcher thread** to accept incoming requests
  - Multiple **worker threads** to process requests
- If a worker blocks (e.g., waiting for disk or I/O), another thread continues execution
- This design:
  - Preserves simple sequential code per request
  - Maximizes resource utilization and parallelism

Single-Threaded Servers

- A single-threaded server processes **one request at a time**.
- The server:
  - Receives a request
  - Processes it fully
  - Sends the reply before accepting the next request
- If the server performs a **blocking operation** (e.g., disk or I/O), it becomes idle.
- This model is:
  - Simple to implement
  - Poor in terms of **performance and scalability**

Event-Driven Servers

- An alternative to threads is an **event-driven server**, implemented as a finite-state machine
- The server:
  - Uses **nonblocking system calls**
  - Records its state when an operation cannot complete
  - Resumes processing when an event occurs
- This model can achieve **high performance and parallelism**
- However, it is:
  - Harder to program
  - More difficult to maintain than threaded designs

Summary: Server Organization Models

Model	Characteristics
Multithreading	Parallelism, blocking system calls
Single-threaded	No parallelism, blocking system calls
Event-driven	Parallelism, nonblocking system calls

Threads allow servers to **retain sequential logic** while achieving concurrency, making them the dominant design choice in distributed systems.

Clients in Distributed Systems

- A **client** is a *process* that **requests services** from one or more servers
- Clients are typically responsible for:
  - Locatings servers
  - Initiating and managing communication
  - Sending requests
  - Receiving and processing replies
- Client behavior strongly influences:
  - Perceived performance
  - Responsiveness
  - Scalability of the distributed system

Servers in Distributed Systems

- A **server** is a *process* that **provides services** to clients
- Servers typically:
  - Wait for incoming requests
  - Execute requested operations
  - Return results to clients
- Servers are usually **long-running processes**
  - Designed to handle many requests over time

Servers in Distributed Systems

- Servers form the **core computational and data resources** of a distributed system
- They are responsible for:
  - Managing shared resources
  - Enforcing consistency and access rules
  - Serving multiple clients concurrently
- Server performance directly affects:
  - System throughput
  - Latency experienced by clients

Servers in Distributed Systems

Design Challenges

- Servers must deal with:
  - **Concurrency**: many clients at the same time
  - **Blocking operations**: disk and network I/O
  - **Scalability**: increasing number of requests
- Poor server design can lead to:
  - Idle CPUs
  - Low request throughput
  - Long response times

Servers in Distributed Systems

Basic Server Execution Model

- A server typically follows this loop:
  1. Wait for a request
  2. Process the request
  3. Send a reply
- If implemented as a **single-threaded process**:
  - A blocking operation suspends the entire server
  - Other clients must wait

Servers in Distributed Systems

Need for Concurrent Servers

- To efficiently serve multiple clients, servers must:
  - Handle requests concurrently
  - Overlap computation and I/O
- This motivates:
  - **Multithreaded servers**
  - **Multiprocess servers**
  - **Event-driven servers**

General principles of concurrency

Process Interaction

In a **multiprogrammed environment**, coexisting processes have different interaction types:

1. **Independent Processes**
   - These processes run independently with no direct communication or data sharing
2. **Cooperating Processes**
   - These processes **share information or explicitly communicate** with each other

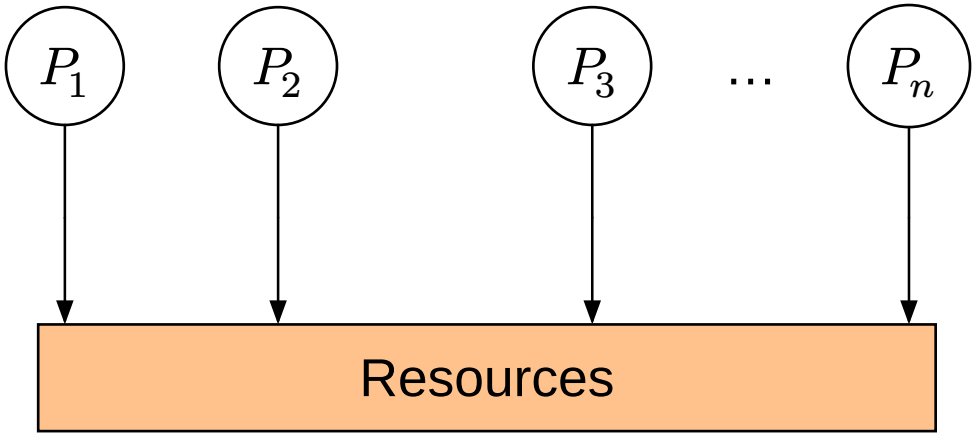
General principles of concurrency

Process Interaction: Independent Processes

- Do not explicitly interact with one another
- Created, executed, and terminated independently
- No direct communication or data sharing
- Implicit interaction only through competition for system resources (CPU, memory, I/O)
- Created by different users, or by the same user for separate tasks
- **Independent processes** compete for shared system resources
  - The **operating system** controls access to prevent data loss
  - Only **one process at a time** may access a resource (**mutual exclusion**)

General principles of concurrency

Process Interaction: Independent Processes



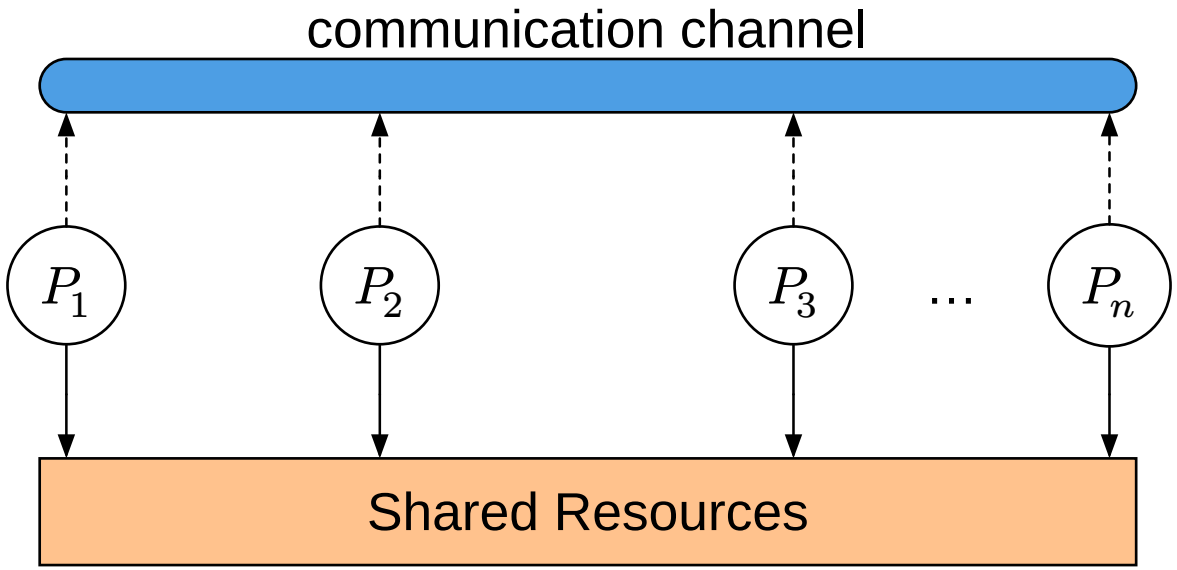
General principles of concurrency

Process Interaction: Cooperating Processes

- **Cooperating processes** share a **common address space** or **shared memory**
  - allows direct access to shared data
- If processes do not share memory:
  - They communicate via **Inter-process communication (IPC)**
    - e.g., pipes, message queues, sockets, or shared memory buffers
- If **Cooperating processes** share data or communicate with each other
  - The **processes themselves** must coordinate access to shared resources
  - **Mutual exclusion** is required to prevent data loss
  - Communication channels are **shared resources** and must be managed accordingly

General principles of concurrency

Process Interaction: Cooperating Processes



Race Conditions

- Occurs when multiple threads **access and modify shared data concurrently**
  - And final outcome depends on the **timing of execution** (i.e. order)
- Threads within the same process **share memory**
  - The operating system does **not protect shared data** between threads.
- If access to shared variables is not properly synchronized:
  - Operations may interleave unpredictably
  - Program behavior becomes **nondeterministic**
- Race conditions are a direct consequence of:
  - Fine-grained concurrency provided by threads
  - The absence of automatic isolation between threads

```java
public class RaceConditions {
    static int counter = 0;
    public static void main(String ...args) throws InterruptedException {
        Runnable task = () -> {
            for (int i = 0; i < 1_000_000; i++)
                counter = counter + 1;
        }; //task
        Thread t1 = new Thread(task); t1.start();
        Thread t2 = new Thread(task); t2.start();
        t1.join(); t2.join();
        System.out.println("Final counter value: " + counter);
        // $ java RaceCondition.java
        // Final counter value: ????
    }
}
```

Critial Region

- Shared resources are accessed through **specific sections of code**
  - Not directly by processes or threads
- When multiple threads execute this code concurrently, **race conditions may occur**
- Uncontrolled interleaving of executions can cause **lost updates and inconsistent state**
  - can result in **race conditions**
- To avoid these problems, access code must be executed **exclusively**
  - i.e.g, **mutual exclusion** is required
- Such protected sections of code are called **critical regions**

Mutual Exclusion Problem

Imposing **mutual exclusion** on access to a resource or a shared region, due to its restrictive nature, can lead to two undesirable consequences:

- **Deadlock / Livelock** – This occurs when two or more processes remain indefinitely **waiting** (either **blocked** or in **busy waiting**) for access to their respective **critical regions**, depending on events that can be **proven never to occur**. The result is the **complete blocking of operations**.
- **Indefinite Postponement (Starvation)** – This occurs when one or more processes compete for access to a **critical region**, but due to an ongoing influx of new competing processes, access is **repeatedly denied**. As a result, the affected processes are **unable to make progress**, creating a real **obstacle to their execution**.

Mutual Exclusion Problem: General Solution

- **Effective enforcement of mutual exclusion** – Access to a **critical region** associated with a given **resource** or **shared region** must be granted to **only one process at a time**, among all processes competing for access **concurrently**.
- **Independence from process execution speed or number** – The correctness of the solution must not rely on any assumptions about **the relative speed of execution** of processes or their **total number**.
- **Non-interference from external processes** – A process **outside** the critical region must **never prevent another process** from entering.
- **No indefinite postponement (Starvation-freedom)** – Any process that **requests access** to a critical region must **eventually be granted entry**, ensuring that access is **not denied indefinitely**.
- **Bounded execution time within a critical region** – A process **inside a critical region** must execute its critical section **within a finite time**, ensuring that **progress is not blocked indefinitely**.

Resources: Definition and Classification

A **resource** is any **component** or **entity** that a process requires for execution.

Resources can be classified into two main categories:

1. **Physical resources** – Hardware components of the computational system, such as:
   - **Processors**
   - **Memory regions** (main memory or mass storage)
   - **Input/output devices** (printers, network interfaces, etc.)
2. **Logical resources** – Shared structures managed by the **operating system** or **application-level processes**, such as:
   - **Process control tables**
   - **Interprocess communication channels**
   - **Shared data structures**

Types of Resource Allocation

A critical property of resources is **how they are appropriated by processes**.

Based on this, resources are categorized as follows:

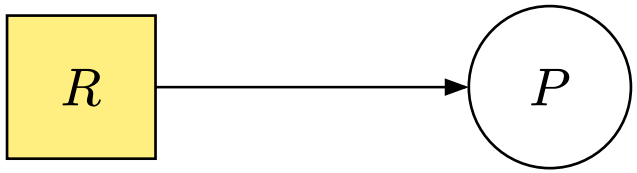
1. **Preemptable resources** – These **can be forcibly reassigned** from one process to another **without causing malfunction**. Examples include:
   - **Processors** (in multiprogramming environments, where execution can be paused and resumed)
   - **Main memory regions** (used to store a process's address space, can be swapped in and out)
2. **Non-preemptable resources** – These **cannot be forcibly reassigned** without causing errors or inconsistencies. Examples include:
   - **Printers** (a print job cannot be interrupted and resumed without corruption)
   - **Shared data structures** (which require **mutual exclusion** for correct access and manipulation)

Deadlock Characterization

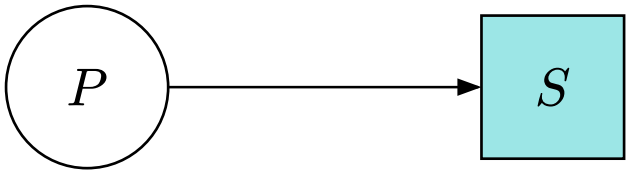
- In a **deadlock**, only **non-preemptable** resources are relevant
- **Non-preemptable** resources cannot be **forcibly reassigned** between processes without risking **malfunction or inconsistency**
- **Preemptable resources** can be **reclaimed** and **reallocated** to let other processes **make progress**, helping **prevent deadlock**

Deadlock Characterization

It is possible to develop a **graphical notation** that **schematically represents deadlock situations**, illustrating the dependencies and potential circular wait conditions that lead to system stagnation.

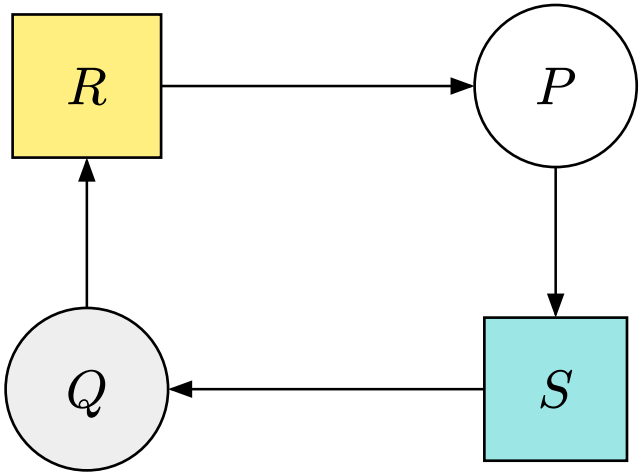


process P holds the resource R



process P requires the resource S

Deadlock Characterization



Typical deadlock situation (the most simplest one)

Necessary Conditions for Deadlock

It can be formally demonstrated that **deadlock** can only occur when all **four necessary conditions** are simultaneously present:

1. **Mutual Exclusion** – Each resource is either **free** or **exclusively assigned** to a single process; resource **holding cannot be shared** among multiple processes.
2. **Hold and Wait (Waiting with Retention)** – A process that **requests a new resource** continues to **hold all previously allocated resources**, instead of releasing them before making new requests.
3. **No Preemption (Non-Liberation)** – A resource that has been **allocated to a process** cannot be **forcibly taken away** by the system; **only the process itself** can decide when to release it.
4. **Circular Wait (Vicious Circle)** – A **circular chain** of processes and resources exists, where **each process in the chain is waiting for a resource currently held by the next process**, forming an inescapable dependency loop.

Deadlock Prevention

The **necessary conditions** for **deadlock occurrence** can be expressed formally as:

$$\text{deadlock occurence} \Rightarrow \begin{array}{l} \text{mutual exclusion } \textbf{and} \\ \text{hold and wait } \textbf{and} \\ \text{no preeemption } \textbf{and} \\ \text{circular wait} \end{array}$$

This statement is **logically equivalent** to its **contrapositive**:

$$\text{no deadlock occurence} \Rightarrow \begin{array}{l} \neg \text{ mutual exclusion } \textbf{or} \\ \neg \text{ hold and wait } \textbf{or} \\ \neg \text{ no preeemption } \textbf{or} \\ \neg \text{ circular wait} \end{array}$$

Deadlock Prevention

- Deadlock can be prevented by **eliminating at least one of its necessary conditions**
- Approaches based on this idea are called **deadlock prevention policies**
- One necessary condition for deadlock is **mutual exclusion**
  - Eliminating mutual exclusion is often **impractical**
    - since it applies to **non-preemptable resources**
  - Allowing shared access to preemptable resources can introduce **race conditions**
  - Race conditions may result in **data inconsistency** and **loss of information**
    - e.g., two threads update a shared bank balance simultaneously; one withdrawal overwrites the other, leaving an incorrect final balance

Deadlock Prevention

- **Deadlock prevention policies** usually avoid **denying mutual exclusion**
- Instead, they focus on eliminating one of the remaining conditions:
  - **hold and wait**
  - **no preemption**
  - **circular wait**

Deadlock Prevention: Deny “hold and wait”

- A process must **request all required resources at once** before execution
- If all resources are **granted immediately**, the process can **execute without interruption**
- If any resource is **unavailable**, it must **wait until all resources become available**
- This strategy **prevents hold-and-wait**, but **does not eliminate starvation**
  - Without additional measures, processes may experience **indefinite postponement**
  - To guarantee progress, systems must enforce **fair resource allocation**
  - **Aging policies** are commonly used to prevent starvation
    – e.g., gradually increasing a waiting process’s priority

Deadlock Prevention: Deny “no preemption”

- **No preemption** means that resources **cannot be forcibly taken** from a process once allocated
- To prevent deadlock by addressing this condition:
  - Processes must **release resources voluntarily** after completing their use
  - **Partial allocation is avoided**: if a requested resource is unavailable, the process **releases all held resources**
  - The process then **waits until all required resources are available** before retrying
- Preempting a **locked or allocated resource** typically requires a **process rollback**
  - Rollback incurs **significant overhead**
  - **preemption-based deadlock prevention** is generally avoided in practice

Deadlock Prevention: Deny “circular wait”

- **Circular wait prevention** is achieved by **imposing a strict linear ordering** on all resource
- Each resource is assigned a **unique numerical order**
- Processes must **request resources in increasing order** of their assigned numbers
- Requests that violate this order are **not permitted**
- **Deadlock cycles cannot form**, since no process can hold a higher-ordered resource while waiting for a lower-ordered one

Resources

- Stalling W., Computer Organization and Architecture: Designing for Performance, 9th Edition, Prentice Hall, 2013
- M. van Steen and A.S. Tanenbaum, Distributed Systems, 4th ed., distributed-systems.net, 2023.